



Universidad Nacional
Autónoma de México
Facultad de Ingeniería



Práctica 03
Disco de Alberty

Asignatura:

Criptografía

Profesor: *Dr. Alfonso Francisco De Abiega L'Eglise*

Grupo: 03

Integrantes - Equipo 10:

Espinoza Matamoros Percival Ulises

García Cortés Adolfo de Jesús

Lugo Manzano Rodrigo

Montiel Juárez Oscar Iván

Fecha de entrega:

8 de Septiembre del 2026

Semestre: 2027-1



Índice

1. Explicación del Método	2
2. Implementación en Python	2
2.1. Generación del anillo móvil	3
2.2. Función de cifrado	3
2.3. Función de descifrado	4
3. Pruebas del Programa	4
4. Repositorio	8
5. Conclusión	8



Práctica 03. Algoritmo de Cifrado: Disco de Alberti

El disco de Alberti, ideado por Leon Battista Alberti en el siglo XV, es considerado el antecesor de los cifrados polialfabéticos. A diferencia del cifrado César, en el que el corrimiento entre el alfabeto plano y el cifrado permanece fijo durante todo el mensaje, el disco de Alberti introduce un mecanismo mecánico: dos anillos concéntricos, uno fijo y otro móvil, que pueden girarse entre sí para cambiar la correspondencia de letras a lo largo del cifrado. Esto dificulta considerablemente el análisis de frecuencias, ya que una misma letra del mensaje original puede cifrarse de distintas formas según el momento en que se gira el disco.

1. Explicación del Método

El disco físico está compuesto por dos anillos con el alfabeto de 26 letras:

- **Anillo fijo (exterior):** representa las letras del mensaje original, siempre en el mismo orden (a, b, c, ..., z).
- **Anillo móvil (interior):** representa las letras cifradas. Al girar el disco, el anillo móvil se desplaza, cambiando la letra con la que queda alineada cada posición del anillo fijo.

Antes de comenzar a cifrar, se define una **letra de alineación inicial**: es la letra del anillo fijo con la que queda alineada la a del anillo móvil al arrancar. A partir de ahí, el disco gira periódicamente conforme se van cifrando letras, usando dos parámetros:

- **Periodo:** cada cuántas letras cifradas se gira el disco.
- **Paso:** cuántas posiciones avanza el anillo móvil cada vez que se gira.

Para cifrar una letra, se busca su posición en el anillo fijo y se toma la letra que le corresponde en el anillo móvil, según el desplazamiento vigente. Cada vez que se cumple el periodo, el desplazamiento aumenta en la cantidad indicada por el paso, simulando el giro físico del disco. Los espacios en blanco se conservan tal cual y no cuentan como letras cifradas, por lo que no provocan que el disco gire.

Para descifrar, el proceso se invierte: dada una letra cifrada, se busca su posición dentro del anillo móvil (con el desplazamiento correspondiente) y se recupera la letra del anillo fijo en esa misma posición. El desplazamiento se recalcula exactamente igual que en el cifrado, por lo que ambas partes deben conocer la letra de alineación inicial, el periodo y el paso para poder comunicarse correctamente.

2. Implementación en Python

El programa modela el anillo fijo como una cadena constante A con el abecedario, y genera el anillo móvil de forma dinámica rotando dicha cadena según el desplazamiento vigente.

2.1. Generación del anillo móvil

```
1 A = "abcdefghijklmnopqrstuvwxyz"
2
3 def obtener_anillo_movil(desplazamiento):
4     """
5     Genera el anillo MOVIL (interior) del disco, rotado segun
6     el desplazamiento actual. Simula "girar" fisicamente el disco.
7     """
8     return A[desplazamiento:] + A[:desplazamiento]
```

La rotación del anillo móvil se logra con una simple concatenación de rebanadas de la cadena: la parte que queda "por delante" del corte se manda al final. Así, si el desplazamiento es 3, el anillo móvil queda como "defghijklmnopqrstuvwxyzabc", que es justo lo que ocurriría si giráramos físicamente el disco tres posiciones.

2.2. Función de cifrado

```
1 def cifrar(mensaje, letra_alineacion, periodo, paso,
2           mostrar_detalle=False):
3     mensaje_cifrado = ""
4     desplazamiento = A.index(letra_alineacion)
5     contador = 0
6
7     for caracter in mensaje:
8         if caracter in A:
9             anillo_movil = obtener_anillo_movil(desplazamiento)
10            indice = A.index(caracter)
11            letra_cifrada = anillo_movil[indice]
12
13            mensaje_cifrado += letra_cifrada
14            contador += 1
15
16            # Girar el disco cada 'periodo' letras cifradas
17            if contador % periodo == 0:
18                desplazamiento = (desplazamiento + paso) % len(A)
19            else:
20                # Los espacios se mantienen y NO giran el disco
21                mensaje_cifrado += caracter
22
23     return mensaje_cifrado
```

El desplazamiento inicial se obtiene ubicando la letra de alineación dentro del anillo fijo A. Por cada letra válida que se cifra, se incrementa un contador independiente del índice del mensaje, de forma que los espacios no alteran el conteo de letras para efectos del giro. Cuando el contador alcanza un múltiplo del periodo, el desplazamiento avanza la cantidad de posiciones indicada por el paso, tomando el módulo 26 para que el disco nunca salga del

rango del abecedario.

2.3. Función de descifrado

```
1 def descifrar(mensaje_cifrado, letra_alineacion, periodo, paso,
2   mostrar_detalle=False):
3     mensaje_descifrado = ""
4     desplazamiento = A.index(letra_alineacion)
5     contador = 0
6
7     for caracter in mensaje_cifrado:
8       if caracter in A:
9         anillo_movil = obtener_anillo_movil(desplazamiento)
10        indice = anillo_movil.index(caracter)
11        letra_original = A[indice]
12
13        mensaje_descifrado += letra_original
14        contador += 1
15
16        if contador % periodo == 0:
17          desplazamiento = (desplazamiento + paso) % len(A)
18        else:
19          mensaje_descifrado += caracter
20
21    return mensaje_descifrado
```

El descifrado utiliza exactamente la misma lógica de giro que el cifrado —mismo desplazamiento inicial, mismo periodo y mismo paso—, pero invierte la búsqueda: en lugar de partir de la posición en el anillo fijo, se busca la letra cifrada dentro del anillo móvil y se recupera la letra correspondiente del anillo fijo en esa posición. Al mantener sincronizados ambos giros, el mensaje original se reconstruye letra por letra sin pérdida de información.

3. Pruebas del Programa

Para validar el algoritmo se cifró el mensaje *“hola equipo diez”* utilizando la letra de alineación inicial c, un periodo de giro de 3 letras y un paso de 5 posiciones. En la Figura 1 se muestra el detalle del cálculo para cada letra durante el cifrado, incluyendo la letra con la que queda alineado el disco en el momento de cifrar cada carácter.

```
--- Calculo del cifrado (letra fija -> disco movil) ---
'h' (fijo pos 7) -> disco alineado en 'c' -> 'j'
'o' (fijo pos 14) -> disco alineado en 'c' -> 'q'
'l' (fijo pos 11) -> disco alineado en 'c' -> 'n'
'a' (fijo pos 0) -> disco alineado en 'h' -> 'h'
'e' (fijo pos 4) -> disco alineado en 'h' -> 'l'
'q' (fijo pos 16) -> disco alineado en 'h' -> 'x'
'u' (fijo pos 20) -> disco alineado en 'm' -> 'g'
'i' (fijo pos 8) -> disco alineado en 'm' -> 'u'
'p' (fijo pos 15) -> disco alineado en 'm' -> 'b'
'o' (fijo pos 14) -> disco alineado en 'r' -> 'f'
'd' (fijo pos 3) -> disco alineado en 'r' -> 'u'
'i' (fijo pos 8) -> disco alineado en 'r' -> 'z'
'e' (fijo pos 4) -> disco alineado en 'w' -> 'a'
'z' (fijo pos 25) -> disco alineado en 'w' -> 'v'
```

Figura 1: Cálculo del cifrado paso a paso, letra de alineación **c**, periodo 3, paso 5.

Puede observarse cómo el disco gira exactamente cada tres letras cifradas: se mantiene alineado en **c** para las primeras tres letras, luego pasa a **h** (avanzando 5 posiciones), después a **m**, a **r** y finalmente a **w**. Los espacios en blanco del mensaje no aparecen en el detalle porque se copian directamente sin consumir un giro del disco.

En la Figura 2 se muestra el proceso inverso, junto con el bloque de resultados finales que confirma que el mensaje descifrado coincide exactamente con el mensaje original.

```
--- Calculo del descifrado (disco movil -> letra fija) ---
'j' (movil alineado en 'c') -> pos 7 -> 'h'
'q' (movil alineado en 'c') -> pos 14 -> 'o'
'n' (movil alineado en 'c') -> pos 11 -> 'l'
'h' (movil alineado en 'h') -> pos 0 -> 'a'
'l' (movil alineado en 'h') -> pos 4 -> 'e'
'x' (movil alineado en 'h') -> pos 16 -> 'q'
'g' (movil alineado en 'm') -> pos 20 -> 'u'
'u' (movil alineado en 'm') -> pos 8 -> 'i'
'b' (movil alineado en 'm') -> pos 15 -> 'p'
'f' (movil alineado en 'r') -> pos 14 -> 'o'
'u' (movil alineado en 'r') -> pos 3 -> 'd'
'z' (movil alineado en 'r') -> pos 8 -> 'i'
'a' (movil alineado en 'w') -> pos 4 -> 'e'
'v' (movil alineado en 'w') -> pos 25 -> 'z'

----- Resultados -----

--> Letra de alineacion inicial:  c
--> Periodo de giro:              3
--> Paso de giro:                 5
--> Mensaje original:             hola equipo diez
--> Mensaje cifrado:              jqnh lxxgubf uzav
--> Mensaje descifrado:           hola equipo diez
```

Figura 2: Cálculo del descifrado y verificación del resultado final.

El mensaje *“hola equipo diez”* se cifra como *“jqnh lxxgubf uzav”*, y al aplicar el descifrado con los mismos parámetros se recupera exactamente el mensaje original, lo que confirma que el giro del disco se mantiene sincronizado en ambas direcciones.

La Figura 3 presenta la ejecución completa del programa. En ella se verifican los parámetros de entrada, el mensaje cifrado y la recuperación correcta del mensaje original al aplicar el descifrado.

```

<<<<<<< Algoritmo de Cifrado - Disco de Alberti <<<<<<<

> Ingrese el mensaje a cifrar: hola equipo diez
> Ingrese la letra con la que inicia alineado el disco (a-z): c
> Ingrese cada cuántas letras gira el disco (ej. 3): 3
> Ingrese cuántas posiciones avanza el disco al girar (1 al 25): 5

--- Cálculo del cifrado (letra fija -> disco móvil) ---
'h' (fijo pos 7) -> disco alineado en 'c' -> 'j'
'o' (fijo pos 14) -> disco alineado en 'c' -> 'q'
'l' (fijo pos 11) -> disco alineado en 'c' -> 'n'
'a' (fijo pos 0) -> disco alineado en 'h' -> 'h'
'e' (fijo pos 4) -> disco alineado en 'h' -> 'l'
'q' (fijo pos 16) -> disco alineado en 'h' -> 'x'
'u' (fijo pos 20) -> disco alineado en 'm' -> 'g'
'i' (fijo pos 8) -> disco alineado en 'm' -> 'u'
'p' (fijo pos 15) -> disco alineado en 'm' -> 'b'
'o' (fijo pos 14) -> disco alineado en 'r' -> 'f'
'd' (fijo pos 3) -> disco alineado en 'r' -> 'u'
'i' (fijo pos 8) -> disco alineado en 'r' -> 'z'
'e' (fijo pos 4) -> disco alineado en 'w' -> 'a'
'z' (fijo pos 25) -> disco alineado en 'w' -> 'v'

--- Cálculo del descifrado (disco móvil -> letra fija) ---
'j' (móvil alineado en 'c') -> pos 7 -> 'h'
'q' (móvil alineado en 'c') -> pos 14 -> 'o'
'n' (móvil alineado en 'c') -> pos 11 -> 'l'
'h' (móvil alineado en 'h') -> pos 0 -> 'a'
'l' (móvil alineado en 'h') -> pos 4 -> 'e'
'x' (móvil alineado en 'h') -> pos 16 -> 'q'
'g' (móvil alineado en 'm') -> pos 20 -> 'u'
'u' (móvil alineado en 'm') -> pos 8 -> 'i'
'b' (móvil alineado en 'm') -> pos 15 -> 'p'
'f' (móvil alineado en 'r') -> pos 14 -> 'o'
'u' (móvil alineado en 'r') -> pos 3 -> 'd'
'z' (móvil alineado en 'r') -> pos 8 -> 'i'
'a' (móvil alineado en 'w') -> pos 4 -> 'e'
'v' (móvil alineado en 'w') -> pos 25 -> 'z'

----- Resultados -----

--> Letra de alineación inicial: c
--> Periodo de giro: 3
--> Paso de giro: 5
--> Mensaje original: hola equipo diez
--> Mensaje cifrado: jqn timer xgubf uzav
--> Mensaje descifrado: hola equipo diez

```

Figura 3: Prueba de ejecución completa del programa del disco de Alberti.



4. Repositorio

El código correspondiente a esta práctica 3 se puede consultar en el siguiente repositorio:

https://github.com/percivalespi/Criptografia/blob/main/practica3/codigo/disco_alberti.py

5. Conclusión

Implementar el disco de Alberti nos permitió entender de forma práctica por qué se le considera el punto de partida de los cifrados polialfabéticos: a diferencia del César y del Vigenère, aquí el corrimiento no depende de una palabra clave letra por letra, sino de un mecanismo de giro periódico que puede ajustarse en frecuencia (periodo) e intensidad (paso). El reto principal fue separar correctamente el conteo de letras cifradas del recorrido del mensaje completo, de modo que los espacios no interfirieran con el momento exacto en que el disco debía girar. Usar el operador módulo sobre la longitud del alfabeto, junto con una variable de contador independiente, nos permitió simular de forma fiel el comportamiento físico del disco y comprobar que el cifrado y el descifrado permanecen perfectamente sincronizados.